

/GET

GET 1 coffee

GET หลาย coffee

The screenshot shows two Postman requests. The left request is a GET to `http://localhost:8080/coffees` with a response body containing a JSON array of two coffee objects. The right request is a GET to `http://localhost:8080/coffees/1` with a response body containing a single coffee object.

```
1 [{"id": 1, "name": "Espresso", "price": 45.0}, {"id": 2, "name": "Latte", "price": 55.0}]
```

```
1 {"id": 1, "name": "Espresso", "price": 45.0}
```

/POST

Post สร้าง coffee ตัวใหม่

เรียกดูผลลัพธ์

The screenshot shows two Postman requests. The left request is a POST to `http://localhost:8080/coffees` with a JSON body for a new coffee. The right request is a GET to `http://localhost:8080/coffees/3` to view the result of the POST.

```
1 {"id": 3, "name": "Macha", "price": 75.0}
```

```
1 {"id": 3, "name": "Macha", "price": 75.0}
```

/PUT

Put แก้ไขข้อมูล coffee

เรียกดูผลลัพธ์

The screenshot shows a REST client interface with two tabs. The left tab is for a PUT request to `http://localhost:8080/coffees/1`. The request body is a JSON object: `{ "id": 1, "name": "Moccha", "price": 45.0 }`. The right tab is for a GET request to the same URL. The response body is the same JSON object. The status bar indicates a 200 OK response with a 3 ms duration.

```
1 {
2   "id": 1,
3   "name": "Moccha",
4   "price": 45.0
5 }
```

```
1 {
2   "id": 1,
3   "name": "Moccha",
4   "price": 45.0
5 }
```

/DELETE

Delete ลบข้อมูล coffee

เรียกดูผลลัพธ์

The screenshot shows a REST client interface with a DELETE request to `http://localhost:8080/coffees/3`. The response body is a JSON array containing two coffee objects: `[ { "id": 1, "name": "Espresso", "price": 45.0 }, { "id": 2, "name": "Latte", "price": 55.0 } ]`. The status bar indicates a 200 OK response with a 3 ms duration.

```
1 [
2   {
3     "id": 1,
4     "name": "Espresso",
5     "price": 45.0
6   },
7   {
8     "id": 2,
9     "name": "Latte",
10    "price": 55.0
11  }
12 ]
```

คำตอบ Discussion 5

HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

ตอบ Get เรียกดูข้อมูล ตามใน project ได้เรียก coffee ที่ id เท่ากับ 1 มาได้ข้อมูลทั้งก้อนของ coffee

Post ขอสร้างข้อมูลตัว coffee จากในโปรเจกต์สร้าง coffee id=3 ชื่อ Macha ราคา 75.0

Put ขอแก้ไขข้อมูล ตามในโปรเจกต์ ได้แก้ไขข้อมูล id=1 เปลี่ยนชื่อเป็น moccha

Delete ขอลบข้อมูล ตามในโปรเจกต์ ขอลบข้อมูล coffee id =1

ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมเมอร์ไม่ทำ

ตอบ เพื่อให้โค้ดอ่านง่ายเป็นระเบียบ จัดการงานแยกกัน

ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้มีไม่หายควรทำอย่างไร (ตอบเป็นแนวคิดพอ)

ตอบ ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนที่โปรแกรมทำเสร็จ ถ้าอยากให้มีไม่หายต้องที่พื้นที่จัดเก็บ เช่น เก็บไว้ในไฟล์, database, cloud

@RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

ตอบ @RestController = กำหนด path แยกให้กับการเรียกข้อมูลหรือแก้ไขข้อมูล

@GetMapping = map ตัว function กับ path ที่จะทำในที่นี้เป็น Get จะใช้กับ เรียกข้อมูล

@PostMapping = map ตัว function กับ path ที่จะทำในที่นี้เป็น Post จะใช้กับ ขอเขียนข้อมูล

@PathVariable = map ค่าของ Parameter ของ path ที่จะทำการยิงให้มาเข้าใจใน java

@RequestBody = รับข้อมูลจาก Body ของ Request แล้วแปลงเป็น Object Java